

8 — State to DOM: only the node that read the value re-runs

SVELTE RUNES FINE-GRAINED

Q. In a button that renders `{claps}` claps, clicking it updates just the number and nothing else on the page. How?

ANSWER. Because the `{claps}` text node is the effect. Reading `claps` inside the markup subscribes that one DOM node to the reactive value. Clicking writes `claps`, the write triggers, and Svelte re-runs only the nodes that read it — a single text update, not a re-render of the component.

WHY IT WORKS. `$state(0)` is the reactive object from the previous step, handed to you as a rune. `{claps}` is the read that subscribes the text node. `claps++` is the write that triggers. So the dependent — and only the dependent — re-runs. Unlike a virtual-DOM framework, where a state change re-runs the whole component function and a diff decides what changed, Svelte tracks the dependency at the node and re-runs just that node.

routes/+page.svelte

```
<script>
  let claps = $state(0);           // a reactive
  value — a signal
</script>

<button onclick={() => claps++}>    // writes
  claps (trigger half)
  {claps} claps                    // reads claps
  (this text node subscribes)
</button>
```

TWO WAYS TO UPDATE

VIRTUAL DOM

`claps++` → re-run whole component → build new tree → diff vs old tree → patch

SVELTE • SURGICAL

`$state(0)`

→

`{claps} node`

`claps++` → only this text node updates

- ① **Summary.** `$state` is the engine we built by hand: the text node that read `claps` is the effect, the click is the trigger, and only that node re-runs. No magic was left — every piece was plain JavaScript.